

Древняя магия в повседневном коде

Пять принципов ФП в пяти языках

Стачка 2026

ФП в повседневном коде

- `[1,2,3].map(x => x * 2)` — функции высшего порядка
- `const add = x => y => x + y` — замыкание, захват лексического окружения
- `const x = 1; x = 2 // TypeError` — запрет перезаписи переменной

ФП — пять конкретных свойств кода

Смысл — **описание результата**, а не микроменеджмент процесса.

Карта доклада

5 свойств × 5 языков

Язык	Философия
OCaml	чистота подхода
Scala	прагматизм JVM
Python	доступность
JavaScript	вездесущность
Rust	системное программирование

Декларативность

Код описывает **что** нужно получить, не **как** это вычислить.

Декларативность

Декларативный код специфицирует *что* вычислить. Императивный — *как* управлять потоком выполнения.

- Паттерн не специфичен для ФП: SQL (`SELECT ... WHERE`), HTML, регулярные выражения — декларативны. ФП — частный случай декларативного стиля.
- `for` -цикл: управление индексом, условием завершения, аккумулятором. `map` / `filter` / `fold` : спецификация трансформации.
- **Зависимость:** декларативный стиль требует функций как значений и иммутабельности данных.

Декларативный код легко оптимизируется: компилятор рассуждает о намерении, а не о конкретных шагах — все применимые преобразования выводимы из семантики

Декларативность — OCaml

Задача: сумма скидок для заказов дороже 100, скидка 10%.

```
(* pipeline c |> *)  
let total_discount orders =  
  let open List in  
  orders  
  |> filter (fun o -> o.price > 100.0)  
  |> map (fun o -> o.price *. 0.1)  
  |> fold_left ( +. ) 0.0
```

Декларативность — Scala

Задача: сумма скидок для заказов дороже 100, скидка 10%.

```
// читается как постановка задачи
def totalDiscount(orders: List[Order]): Double =
  orders
    .filter(_.price > 100.0)
    .map(_.price * 0.1)
    .sum
```

Декларативность — Rust

Задача: сумма скидок для заказов дороже 100, скидка 10%.

```
fn total_discount_imp(orders: &[Order]) -> f64 {  
    let mut total = 0.0;  
    for o in orders {  
        if o.price > 100.0 { total += o.price * 0.1; }  
    }  
    total  
}
```

```
fn total_discount(orders: &[Order]) -> f64 {  
    orders.iter()  
        .filter(|o| o.price > 100.0)  
        .map(|o| o.price * 0.1)  
        .sum()  
}
```

// LLVM компилирует обе версии в идентичный машинный код

Декларативность — Python

Задача: сумма скидок для заказов дороже 100, скидка 10%.

```
def total_discount_imp(orders):  
    total = 0.0  
    for o in orders:  
        if o.price > 100.0:  
            total += o.price * 0.1  
    return total  
  
def total_discount(orders):  
    return sum(o.price * 0.1 for o in orders if o.price > 100.0)
```

Декларативность — JavaScript

Задача: сумма скидок для заказов дороже 100, скидка 10%.

```
function totalDiscountImp(orders) {  
  let total = 0;  
  for (const o of orders) {  
    if (o.price > 100) total += o.price * 0.1;  
  }  
  return total;  
}
```

```
const totalDiscount = (orders) =>  
  orders  
    .filter((o) => o.price > 100)  
    .map((o) => o.price * 0.1)  
    .reduce((acc, d) => acc + d, 0);
```

Выражения вместо инструкций

Если `if` возвращает значение — код становится компонуемым.

Выражения вместо инструкций

Инструкция выполняется ради эффекта. *Выражение* вычисляется в значение. В expression-oriented языках всё — выражение.

- **Компонуемость:** выражение подставляется в любую позицию, ожидающую значение. `f(if x > 0 then a else b)` корректно в OCaml / Scala / Rust; требует ternary-оператора в Python / JS.
- **Exhaust checking:** OCaml и Rust статически верифицируют полноту `match`. Пропущенная ветка — ошибка компиляции.
- **Pattern matching:** деструктуризация + связывание переменных + exhaust checking в одной конструкции.

	OCaml	Scala	Rust	Python	JS
<code>if</code> — выражение	✓	✓	✓	—	—

Выражения вместо инструкций — OCaml

Задача: классификация числа — вернуть метку без промежуточной переменной.

```
let classify n =  
  match n with  
  | n when n < 0 -> "negative"  
  | 0             -> "zero"  
  | _             -> "positive"  
  
let label = String.uppercase_ascii (classify (-5))  
(* выражение в позиции аргумента *)
```

Выражения вместо инструкций — Scala

Задача: классификация числа — вернуть метку без промежуточной переменной.

```
def classify(n: Int): String =  
  n match {  
    case n if n < 0 => "negative"  
    case 0          => "zero"  
    case _          => "positive"  
  }  
  
val label = classify(-5).toUpperCase
```

Выражения вместо инструкций — Rust

Задача: классификация числа — вернуть метку без промежуточной переменной.

```
fn classify(n: i32) -> &'static str {  
    match n {  
        n if n < 0 => "negative",  
        0          => "zero",  
        _          => "positive",  
    }  
}  
  
let label = classify(-5).to_uppercase();
```

Выражения вместо инструкций — Python

Задача: классификация числа — вернуть метку без промежуточной переменной.

```
def classify(n: int) -> str:  
    return (  
        "negative" if n < 0 else  
        "zero"     if n == 0 else  
        "positive"  
    )
```

```
label = classify(-5).upper()
```

```
# label = match n: ... → SyntaxError: match — инструкция
```


Выражения вместо инструкций — JavaScript

Задача: классификация числа — вернуть метку без промежуточной переменной.

```
const classify = (n) =>  
  n < 0 ? "negative"  
  : n === 0 ? "zero"  
  : "positive";
```

```
const label = classify(-5).toUpperCase();
```

```
// if (...) { ... } → инструкция, нет значения  
// n < 0 ? ... : ... → выражение, есть значение
```

Функции как значения

First-class value: передаётся аргументом, связывается с переменной, возвращается из функции.

Функции как значения

Функция — *first-class value*: связывается с переменной, передаётся аргументом, возвращается из функции.

- **HOF** (*higher-order function*) — принимает или возвращает функцию. `map`, `filter`, `fold` — частный случай.
- **Замыкание** — функция с захватом лексического окружения. Rust: явный `move`; JS: неявный захват.
- **Каррирование** — `f(a, b) → f(a)(b)`. OCaml: по умолчанию; Python: `functools.partial`; JS: явные замыкания.

Свойство поддерживается во всех пяти языках. Различие — степень **глубины интеграции**.

Функции как значения — OCaml

Задача: создать умножитель с коэффициентом и применить к списку.

```
(* каррирование по умолчанию *)
let multiply factor x = x * factor
let triple = multiply 3

(* замыкание: base захвачен в лексическом окружении *)
let make_adder base = fun x -> base + x
let add10 = make_adder 10 (* add10 5 = 15 *)

(* HOF: функция как аргумент *)
let result = List.map triple [1; 2; 3; 4; 5]
(* [3; 6; 9; 12; 15] *)
```

Функции как значения — Scala

Задача: создать умножитель с коэффициентом и применить к списку.

```
// каррирование через .curried
val multiply: (Int, Int) => Int = (factor, x) => x * factor
val triple: Int => Int = multiply.curried(3)

// замыкание: base захвачен в лексическом окружении
val base = 10
val addBase: Int => Int = x => x + base

val result = List(1, 2, 3, 4, 5).map(triple)
// List(3, 6, 9, 12, 15)
```

Функции как значения — Rust

Задача: создать умножитель с коэффициентом и применить к списку.

```
use auto_curry::curry;

#[curry]
fn multiply(factor: i32, x: i32) -> i32 { factor * x }

let triple: impl Fn(i32) -> i32 = multiply(3);

let base = 10;
let add_base: impl Fn(i32) -> i32 = move |x| x + base;

let result: Vec<i32> =
    vec![1, 2, 3, 4, 5].into_iter().map(triple).collect();
```

Функции как значения — Python

Задача: создать умножитель с коэффициентом и применить к списку.

```
from functools import partial

def multiply(factor, x): return factor * x
triple = partial(multiply, 3) # каррирование

result = list(map(triple, [1, 2, 3, 4, 5]))
# [3, 6, 9, 12, 15]

# декоратор = HOF: fn → fn
def logged(fn):
    def wrapper(*a): return fn(*a)
    return wrapper

@logged
def triple2(x): return x * 3
```

Функции как значения — JavaScript

Задача: создать умножитель с коэффициентом и применить к списку.

```
import * as R from "ramda";

const multiply = R.curry((factor, x) => factor * x);
const triple = multiply(3);

const base = 10;
const addBase = (x) => x + base;

const result = [1, 2, 3, 4, 5].map(triple);
// [3, 6, 9, 12, 15]
```


Ссылочная прозрачность

$f(x)$ всегда возвращает одно и то же для одного x .

Ссылочная прозрачность

Выражение *ссылочно прозрачно*, если его можно заменить значением без изменения поведения программы.

- **Чистая функция:** возвращаемое значение зависит исключительно от аргументов; нет побочных эффектов.
- **Следствия:** корректная мемоизация; независимость порядка вычислений; data race-free параллелизм.
- **Эффекты:** неизбежны (I/O, время, состояние). Стратегия — **изоляция** на границах системы.

Язык	Подход к чистоте
OCaml	чистота — умолчание; <code>ref</code> — явное исключение
Scala	эффекты в типе: <code>IO[Double] ≠ Double</code>

Ссылочная прозрачность — OCaml

Задача: функция скидки — чистая и нечистая версии.

```
let current_discount = ref 0.1

let price_with_global_discount price =
  price *. (1.0 -. !current_discount)

let apply_discount rate price =
  price *. (1.0 -. rate)

let discounted = apply_discount 0.1 100.0
```

Ссылочная прозрачность — Scala

Задача: функция скидки — чистая и нечистая версии.

```
def applyDiscount(rate: Double, price: Double): Double =  
    price * (1.0 - rate)  
  
def withLog(rate: Double, price: Double): IO[Double] =  
    IO.println("...") *> IO.pure(applyDiscount(rate, price))
```

Ссылочная прозрачность — Rust

Задача: функция скидки — чистая и нечистая версии.

```
fn apply_discount(rate: f64, price: f64) -> f64 {  
    price * (1.0 - rate)  
}  
  
// нечистая версия — &mut в сигнатуре  
fn apply_discount_mut(rate: f64, price: &mut f64) {  
    *price *= 1.0 - rate;  
}
```

Ссылочная прозрачность — Python

Задача: функция скидки — чистая и нечистая версии.

```
current_discount = 0.1

def price_with_global_discount(price: float) -> float:
    return price * (1 - current_discount)

def apply_discount(rate: float, price: float) -> float:
    return price * (1 - rate)
```

Ссылочная прозрачность — JavaScript

Задача: функция скидки — чистая и нечистая версии.

```
let taxRate = 0.2;  
const priceWithTax = (price) => price * (1 + taxRate);  
  
const applyDiscount = (rate) => (price) => price * (1 - rate);
```

Иммутабельность

Данные не изменяются — создаются новые версии.

Иммутабельность

`const` / `val` — запрет перезаписи переменной. Иммутабельность — запрет мутации самого значения.

- **Гарантия компилятором** (OCaml `let`, Rust `let`) / **соглашением** (Scala `val`) / **опционально** (Python `frozen=True`, JS `Object.freeze`)
- **Persistent data structures: *structural sharing***. Обновление одного поля — $O(\log n)$, не $O(n)$.
- **Следствия:** нет data races; нет defensive copies; referential transparency по конструкции.

Иммутабельность — OCaml

Задача: обновить одно поле записи без изменения оригинала.

```
(* record иммутабелен *)
type user = {
  name: string;
  age: int;
}

let user = { name = "Alice"; age = 30 }

let older = { user with age = 31 }
(* user не изменился *)
```

Иммутабельность — Scala

Задача: обновить одно поле записи без изменения оригинала.

```
case class User(name: String, age: Int)

val user = User("Alice", 30)
val older = user.copy(age = 31)
// user.age = 31 → error: reassignment to val
```

Иммутабельность — Rust

Задача: обновить одно поле записи без изменения оригинала.

```
struct User { name: String, age: u32 }

let user = User { name: "Alice".into(), age: 30 };
let older = User { age: 31, ..user };

let mut draft = User { name: "Bob".into(), age: 25 };
draft.age = 26; // только let mut допускает мутацию
```

Иммутабельность — Python

Задача: обновить одно поле записи без изменения оригинала.

```
from dataclasses import dataclass, replace

@dataclass(frozen=True)
class User:
    name: str
    age: int

user = User("Alice", 30)
older = replace(user, age=31)
user.age = 31 # FrozenInstanceError: cannot assign to field 'age'
```

Иммутабельность — JavaScript

Задача: обновить одно поле записи без изменения оригинала.

```
"use strict";

const user = Object.freeze({ name: "Alice", age: 30 });
const older = { ...user, age: 31 };

user.age = 31; // TypeError: Cannot assign to read only property
```

Итог: пять свойств x пять языков

Свойство	OCaml	Scala	Rust	Python	JS
Декларативность	+++	+++	+++	++	++
Выражения	+++	++	+++	+	+
Функции как значения	+++	+++	++	++	+++
Ссылочная прозрачность	+++	++	++	+	+
Иммутабельность	+++	++	+++	+	+

+++ — в дизайне языка, идиоматично ++ — поддерживается, требует дисциплины + — возможно, но нетипично

Выводы

ФП — набор формализуемых свойств кода, применимых независимо от языка и декларируемой парадигмы.

Вопросы:

1. Какие из пяти свойств присутствуют в вашем текущем проекте?
2. Какие из них язык гарантирует системой типов, а какие — только конвенцией?
3. Где явная иммутабельность или декларативный стиль снизили бы когнитивную нагрузку в последнем PR?

Спасибо

Стачка 2026

Материалы доклада, sandbox и примеры кода: